Assignment 10 — Audio Classification with STT and TTS

Deep Learning | Practical Report / Solution

BRACT's Vishwakarma Institute of Technology, Pune

Department: CSE-AI **Semester:** 5 **Academic Year:** 2026-27

Division: E **Roll No.:** 64 **PRN:** 12412320 **Name:** Samyak Shende

Problem Statement

Build a deep-learning application that classifies audio commands and demonstrates Speech-to-Text (STT) and Text-to-Speech (TTS), evaluating the relevant tasks with suitable metrics.

Objective

Train a CNN to classify spoken commands from audio, integrate Whisper for speech recognition, and use gTTS to synthesize speech from text. Evaluate audio classification with standard metrics and demonstrate an end-to-end audio workflow.

Dataset

The notebook downloads the Mini Speech Commands dataset. Audio recordings are organized into command-class folders. Up to 1,200 files per class are selected (`MAX_PER_CLASS = 1200`). The notebook uses a stratified split: 70% training, 15% validation, and 15% testing.

Audio Preprocessing and Feature Extraction

Audio is converted to mono, resampled to 16 kHz, and padded or truncated to one second. Each waveform is transformed into a 64-band log-Mel spectrogram using an FFT size of 512 and hop length of 256. The resulting spectrograms are normalized and supplied to the CNN with a single input channel.

CNN Architecture and Training

The CNN contains three convolutional blocks with 32, 64, and 128 filters. Each block uses ReLU activation, batch normalization, and max pooling. Dropout is applied before global average pooling and a 128-unit dense layer. The output layer uses softmax for command-class prediction. The model is trained with Adam (learning rate 0.001), sparse categorical cross-entropy, up to 25 epochs, and batch size 64. Early stopping, learning-rate reduction, and best-model checkpointing are used.

Speech-to-Text (STT)

The notebook initializes the pretrained `openai/whisper-tiny.en` model through the Transformers automatic-speech-recognition pipeline. Whisper transcribes the uploaded audio file when initialization and inference succeed. The notebook does not calculate Word Error Rate (WER); STT is demonstrated through the generated transcript.

Text-to-Speech (TTS)

The notebook uses gTTS to convert supplied text into an MP3 file named `tts_output.mp3` and plays the generated audio in the notebook. An internet connection is required for gTTS.

End-to-End Workflow

A user uploads a WAV or MP3 recording. The audio is passed independently to Whisper for transcription and to the CNN for command classification. The CNN returns the predicted command, confidence, and top-three class predictions. The notebook can then use TTS to speak text, demonstrating the STT, classification, and TTS components together.

Evaluation Metrics

Task	Evaluation
Audio command classification	Test accuracy, weighted precision, weighted recall, weighted F1-score, classification report, confusion matri
Speech-to-Text	Transcript demonstration; WER is not computed in the supplied notebook

Task	Evaluation
Text-to-Speech	Functional generation and playback of an MP3; no objective TTS quality metric is implemented

Results

The uploaded notebook contains code to calculate and export the test metrics to `audio_classification_metrics.csv`, but no executed metric outputs were supplied. Therefore, this report does not invent test accuracy, precision, recall, F1-score, confusion-matrix values, or a sample transcription. Run the notebook to obtain the actual results.

Conclusion

The implementation combines three components: CNN-based spoken-command classification using log-Mel spectrograms, Whisper-based speech transcription, and gTTS-based speech generation. It includes a reproducible dataset split, model training callbacks, classification evaluation, and an uploaded-audio demonstration. The classification metrics must be recorded from an actual notebook run. STT and TTS are functional demonstrations in this implementation rather than fully benchmarked tasks.

Execution Notes

The notebook is designed for Google Colab. Internet access is required to download the dataset, initialize Whisper, and use gTTS. A GPU is helpful but optional. The uploaded-audio step uses Google Colab's file-upload interface.